Actividad 3.2 — Diseño de un pipeline seguro

Cuidarte+: descarga de PDF de exámenes por paciente

Integrantes: Benjamín Ibáñez y Gerardo Vera Fecha: 3 de noviembre de 2026

Este documento presenta el pipeline que revisaría el cambio antes de liberarlo. Es solo el diseño: no se instaló nada ni se ejecutó el pipeline.

1. Sistema y cambio elegido

Sistema: Cuidarte+, la plataforma de centros de salud para administrar pacientes y exámenes médicos (roles: administrador, médico y paciente).

Cambio: que el paciente pueda descargar en PDF el resultado de sus propios exámenes. El backend agrega un endpoint que recibe el id del examen, comprueba que es de ese paciente y entrega el PDF.

Qué está en riesgo: los exámenes son datos de salud (datos sensibles, Ley N° 19.628). Lo más delicado del cambio es el control de acceso: el servidor debe comprobar en cada descarga que el examen pertenece al paciente que lo pide. Si no lo hace, cualquiera podría ver exámenes ajenos con cambiar un número en la dirección (el mismo error del caso First American, Actividad 3.1). También está en riesgo la descarga misma: si falla, el paciente no puede acceder a sus exámenes.

2. Diagrama del pipeline

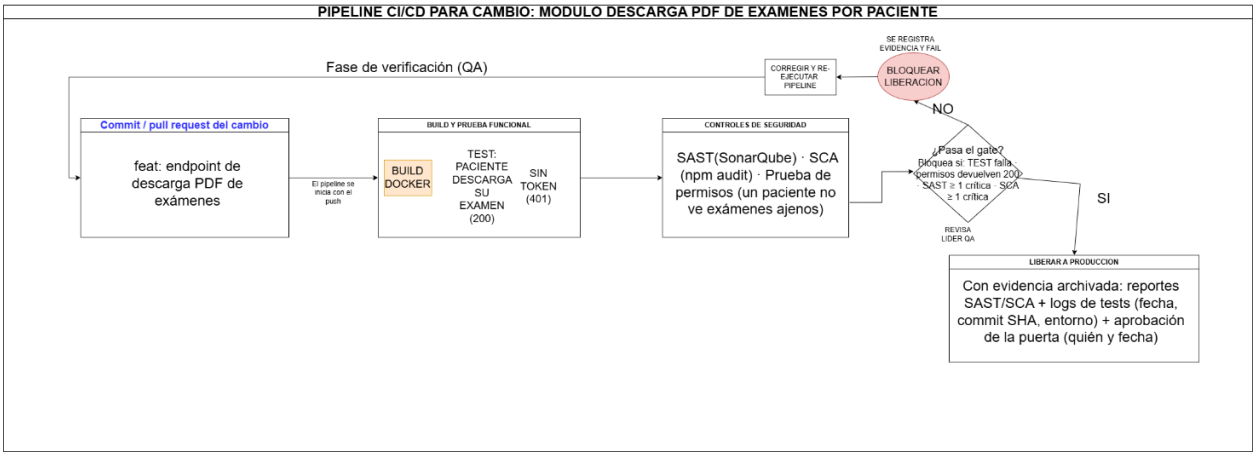


Figura 1. Pipeline propuesto para el cambio.

El flujo, en simple, es este:

- **Commit / pull request.** El desarrollador sube el cambio y el pipeline se gatilla solo.
- **Build y prueba funcional.** Se construye la imagen Docker y corren las pruebas: descarga con token del dueño → 200 y PDF válido; sin token → 401.
- **Controles de seguridad.** Tres en fila: SAST (SonarQube), SCA (npm audit) y una prueba de permisos (un paciente intenta descargar el examen de otro).
- **Puerta de calidad.** Decide si se libera o no (regla en la sección 4).

- **Producción.** Solo si la puerta aprueba, se libera guardando la evidencia. Si no aprueba, se bloquea, se corrige y se repite todo el pipeline.

3. Qué revisa cada control

El pipeline tiene una prueba funcional y tres controles de seguridad.

Control	¿Qué revisa?	¿Qué riesgo detecta?	¿Qué entrega al equipo?
Prueba funcional (descarga del PDF)	Que la descarga funcione: 200 con un PDF válido cuando el dueño del examen la pide, y 401 si no hay token.	Que el cambio rompa algo que ya andaba, o que la autenticación esté ausente o mal aplicada.	Log de las pruebas con PASS/FAIL por caso y el código HTTP obtenido vs. el esperado.
SAST — SonarQube	El código fuente del backend, sin ejecutarlo: rutas, validaciones y generación del PDF. Busca patrones peligrosos (inyección SQL, XSS, secretos en el código, etc.).	Fallas escritas por nosotros en el propio cambio, antes de que lleguen a producción.	Reporte de incidencias clasificadas por severidad (críticas, mayores, menores), con regla y ubicación (archivo:línea).
SCA — npm audit	Las dependencias del proyecto (Express, pg, multer, yup…) contra bases públicas de fallos conocidos.	Paquetes de terceros con vulnerabilidades conocidas. El caso MOVEit (Actividad 3.1) fue exactamente eso: un producto con una falla conocida.	Listado de paquetes con fallo, severidad, identificador (CVE) y versión que corrige; código de salida distinto de cero sobre el umbral definido.
Prueba de permisos	Que el servidor compruebe en cada descarga que el examen es del paciente del token. El paciente A pide el examen del paciente B: la única respuesta aceptable es un rechazo.	Que un paciente vea datos clínicos de otro cambiando el id del examen (el mismo patrón del caso First American). Es el riesgo más grave del cambio.	PASS/FAIL: el examen ajeno debe responder 403 (o 404), nunca 200. Se guarda evidencia de la petición y la respuesta.

4. Puerta de calidad

¿Qué bloquea la liberación? Basta cualquiera de estas condiciones:

- Falla la prueba funcional: la descarga no responde 200, o sin token no responde 401.
- Falla la prueba de permisos: un paciente consigue descargar el examen de otro (200).
- SAST reporta al menos una vulnerabilidad crítica.
- SCA reporta una dependencia crítica sin versión que la corrija aplicada.

¿Quién revisa? El líder técnico (o el encargado de QA) revisa la evidencia junto al responsable del cambio. El autor no aprueba su propio cambio.

¿Qué evidencia se guarda? El reporte de SonarQube, la salida de npm audit, el log de las pruebas (fecha, commit y entorno) y la decisión de la puerta: aprobada o bloqueada, quién la revisó y cuándo. Todo queda archivado junto al commit.

Si bloquea: no se libera. Se corrige y se repite todo el pipeline, no solo la etapa que falló, porque un cambio puede romper controles que ya estaban en verde.

5. Los dos escenarios de la guía

A. La prueba funcional pasa, pero un usuario accede a información de otra persona

Decisión: no se libera.

Que la función responda no significa que responda solo a quien debe. Acá el problema es de acceso: alguien ve datos de salud ajenos, que son datos sensibles. Es el mismo error del caso First American (Actividad 3.1). Liberar «y arreglarlo después» expondría datos de pacientes reales. Lo que se hace: corregir el control de acceso en el servidor, agregar ese caso como prueba fija del pipeline, repetir todo el pipeline y, si el código ya estaba en un ambiente accesible, evaluar a qué datos se accedió y avisar según el protocolo del equipo.

B. El análisis de dependencias no se ejecutó por un error de configuración

¿Qué se sabe? Solo que esa etapa no corrió. De las dependencias no se sabe nada: ni si tienen fallos conocidos, ni si el cambio agrega paquetes vulnerables.

¿Qué no se sabe? Si es seguro liberar respecto de dependencias. Una etapa que no se ejecutó no es un aprobado: «sin alertas» y «sin analizar» son cosas distintas.

¿Qué haría antes de liberar? No liberar todavía. Corregir la configuración y repetir el pipeline. Si no se puede a tiempo, correr npm audit a mano sobre el mismo commit, adjuntar el resultado y pedir la aprobación escrita del líder técnico. Sin una de esas dos cosas, la puerta no pasa.

6. Preguntas de cierre

¿Qué ventaja tiene automatizar los controles? Que se repiten igual en cada cambio, en minutos, y queda registro de todo. No dependen de que alguien se acuerde de revisarlos.

¿Por qué sin alertas no significa que el sistema sea seguro? Porque cada control solo ve lo suyo: el SAST no entiende las reglas de negocio, el SCA solo conoce fallos ya publicados y las pruebas solo cubren los casos que alguien escribió. «Sin alertas» quiere decir «sin hallazgos en lo revisado», no «sin riesgo».